Instruction-Conditioned Adapter：Instructionに基づく動的制御によるプロンプトインジェクション防御

米田 智紀[1,a)]　大塚 玲[1,b)]

概要：信頼できない外部データを処理する LLM では，間接プロンプトインジェクションが重要な課題となる．既存手法は Instruction と Data に異なる埋め込み変換を適用することで両者の表現を分離するが，依然として有用性とセキュリティの両立や，Data 内の命令の影響を十分に抑制することは依然として困難であり，データ内の指示によって本来の指示が上書きされる問題が残されている．本研究では，各 Transformer 層で Instruction の表現を集約し，Data および Response に対する Adapter の出力を Instruction に応じて制御する Instruction-Conditioned Adapter（ICA）を提案し，ASIDE と組み合わせることで，Instruction と Data を区別しつつ Instruction に応じた処理を実現する．構造変更による効果を確認するため，同一の Alpaca-clean-gpt4-turbo データセットおよび学習条件で学習し，既存手法である ASIDE および Instructional Segment Embedding（ISE），AIR と比較する．固定のプロンプトインジェクション攻撃に対する評価では，ASIDE の平均攻撃成功率が 9.60%であったのに対し，ASIDE+ICA では 7.81%となった．さらに，PAIR を用いた適応的プロンプトインジェクション評価では，ASIDE が 60.58%であったのに対し，ASIDE+ICA では 41.35%となり，ASIDE と比較して 19.23 ポイント低減した．これらの結果から，Instruction と Data の区別に加えて，Instruction の内部表現に基づいて Data および Response に対する処理を動的に制御することが，プロンプトインジェクションに対する頑健性向上に寄与し，特に適応的攻撃に対して有効である可能性が示唆される．

キーワード：プロンプトインジェクション，LLM セキュリティ，適応的攻撃

Instruction-Conditioned Adapter: Defending Prompt Injection via Instruction-Conditioned Control

Tomonori Yoneda[1,a)]　Akira Otsuka[1,b)]

Abstract: Indirect prompt injection remains a critical challenge for LLMs that process untrusted external data. Existing defenses separate the representations of Instruction and Data by applying different embedding transformations to each. However, simultaneously maintaining utility and security remains difficult, and the influence of instructions embedded in Data cannot always be sufficiently suppressed, leaving the original Instruction vulnerable to being overridden by instructions contained in the Data. In this work, we propose the Instruction-Conditioned Adapter (ICA), which aggregates the representation of the Instruction at each Transformer layer and controls the output of the Adapter applied to Data and Response according to the Instruction. By combining ICA with ASIDE, our approach distinguishes Instruction from Data while enabling Instruction-conditioned processing. To evaluate the effect of this architectural modification, we train models using the same Alpaca-clean-gpt4-turbo dataset and training conditions, and compare them with existing methods, including ASIDE, Instructional Segment Embedding (ISE), and AIR. Under fixed prompt injection attacks, ASIDE achieves an average attack success rate of 9.60%, whereas ASIDE+ICA reduces it to 7.81%. Furthermore, under adaptive prompt injection attacks using PAIR, the attack success rate of ASIDE is 60.58%, while ASIDE+ICA reduces it to 41.35%, corresponding to a 19.23 percentage-point reduction. These results suggest that, in addition to distinguishing Instruction from Data, dynamically controlling the processing of Data and Response based on the internal representation of the Instruction can improve robustness against prompt injection, with particularly pronounced benefits against adaptive attacks.

Keywords: Prompt Injection, LLM security, Adaptive Attack

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

大規模言語モデル（Large Language Model; LLM）は，検索結果，Web ページ，メール，文書，ツール出力などの外部データを参照しながら，ユーザの要求に応じた処理を行う AI エージェントへ急速に組み込まれている．一方，これらの外部データは必ずしも信頼できない．攻撃者が外部データ中に悪意ある命令を埋め込むことで，モデルに本来従うべき命令とは異なる挙動を取らせる間接プロンプトインジェクションは，LLM を外部環境と連携させる上で重要な脅威となっている [1].

本研究では ASIDE [4] に従い，モデルが実行すべき命令を Instruction，Instruction に従って処理される外部情報を Data とし，Instruction はシステムまたは正規ユーザから与えられ，攻撃者によって直接変更されないものとする．一方，Data は検索結果，Web ページ，メール，文書，ツール出力などの外部情報を含み，攻撃者によって操作される可能性があるものとする．

プロンプトインジェクションに対しては， Instruction と Data に異なる区切りやタグを付与して 入力形式を構造化する手法や， 両者の区別や優先度を embedding や中間表現などのモデル内部に組み込む手法が提案されている [2], [3], [4], [5]． これらの手法は， Instruction と Data を明示的に区別することで， Data 中の攻撃命令が正規の Instruction として扱われることを抑制する．

しかし，これらの既存手法には，Data の扱いを現在の Instruction の内容に応じて変化させることができないという課題がある．ISE，ASIDE，AIR などは，Instruction と Data の区分や優先度をモデル内部に与えることで両者を区別するが，その制御は Instruction が具体的に何を要求しているかには依存しない [3], [4], [5]．そのため，同一の Data に対しては，Instruction の内容が異なる場合でも基本的に同じ防御処理が適用される．

この課題は，Data に対して求められる処理が Instruction によって異なる場合に顕在化する．例えば，Data 中に命令形式の文が含まれている場合，要約タスクではその文を実行せず文章の一部として扱う必要があるが，命令文を抽出するタスクでは重要な処理対象となる．このように，Data 中の情報の適切な扱いは Instruction との関係によって変化する．したがって，固定的な区分や変換だけでは，タスクに必要な情報を保持しながら Data 中の攻撃命令の影響を抑制することが難しい可能性がある．

そこで本研究では，現在の Instruction が何を要求しているかという意味情報を Data の内部処理に反映することに着目し，Instruction-Conditioned Adapter（ICA）を提案する．ICA は，既存の Instruction-Data 分離機構そのものを変更するのではなく，Instruction の意味に基づく動的な内部制御を追加することで，既存の分離機構を補完する防御機構である．

具体的には，ICA は各 Decoder 層において Instruction の内部表現を集約し，その表現からゲートを生成する．生成されたゲートを用いて，Data および Response 側の内部処理を調整することで，現在の Instruction の内容に応じた動的な制御を実現する．

ICA は特定の Instruction-Data 分離手法に依存せず，既存の分離機構と組み合わせて利用することを想定している．本研究では，その具体例として ASIDE に ICA を適用し，その有効性を検証する．

本稿の貢献は以下の 2 点である．

- Instruction の意味表現から各層のゲートを生成し，Data および Response 側の内部処理を Instruction の内容に応じて動的に制御する Instruction-Conditioned Adapter（ICA）を提案する．
- Vanilla, ISE, ASIDE, AIR, ICA-Only, および ASIDE に ICA を適用したモデルを比較し，StruQ のプロンプトインジェクション評価および PAIR を用いた評価を通じて，プロンプトインジェクション耐性と通常のタスク性能に与える影響を検証する．

2. 関連研究

プロンプトインジェクションが成立する要因の一つとして，LLM がモデルの従うべき Instruction と，処理対象である Data を十分に区別できないことが指摘されている [2]．この問題に対して，Instruction と Data を入力形式上で明示的に分離する方法や，入力の役割や優先度をモデル内部に与えることで，モデルが従うべき Instruction を明確にする方法が提案されている．StruQ [2] は，Instruction と Data を異なる領域として明示する Structured Query を導入した．専用の区切りを用いて両者を入力形式上で分離するとともに，Data 中に命令文が含まれる場合でも Instruction 側の命令を優先するようモデルを学習することで，プロンプトインジェクションへの耐性を向上させる．

StruQ の考え方を発展させた手法として，SecAlign [6] が提案されている．SecAlign は，本来の Instruction に従った応答を望ましい出力，Data 中の攻撃命令に従った応答を望ましくない出力として選好データを構成し，Direct Preference Optimization（DPO） [13] によって前者を優先するようモデルを調整する．さらに，Meta SecAlign [7] では，SecAlign の枠組みを発展させ，一般的な指示追従タスクに加えて，ツール利用や Web 操作を含むエージェント環境におけるプロンプトインジェクション耐性についても評価している．また，複数の Instruction が競合する状

[1] 情報セキュリティ大学院大学
Institute of Information Security
[a)] mgs211101@iisec.ac.jp
[b)] otsuka@iisec.ac.jp

© 2026 Information Processing Society of Japan

況に対し，Instruction 間の優先順位を明示的に定める考え方として，Instruction Hierarchy [8] が提案されている．Instruction Hierarchy では，System，User，外部 Data など，異なる入力源から与えられる Instruction に優先度を設定し，複数の命令が競合した場合には，より高い優先度を持つ Instruction に従うようモデルを学習する．これにより，優先度の低い入力に含まれる命令によって，より優先度の高い Instruction が上書きされることを抑制する．

これらの入力形式による区別や Instruction の優先順位付けに加え，Instruction と Data の区別をモデル内部の表現や構造に直接組み込む手法も提案されている．Wu らは Instructional Segment Embedding（ISE）[3] を提案し，各 token が属する入力区分や優先度を表す学習可能な embedding を token embedding に加えることで，Instruction Hierarchy をモデル内部に明示した．これにより，Instruction，Data，Response などの役割を入力表現の段階から区別できるようにする．

一方，Instruction Hierarchy を表す情報は主として入力層において付与される．Zverev らは Architecturally Separated Instruction-Data Embeddings（ASIDE） [4] を提案した．ASIDE は，Data token の embedding に固定された直交回転を適用することで，Instruction と Data を embedding 空間上で構造的に分離する．この変換自体に追加の学習可能なパラメータを必要とせず，Instruction と Data の区別をモデルアーキテクチャに直接組み込める点を特徴とする．また，通常の指示追従能力を維持しながら，Instruction–Data separation およびプロンプトインジェクションに対する頑健性を向上できることが報告されている．

Kariyappa らは Augmented Intermediate Representations（AIR）[5] を提案した．AIR は，Instruction の優先度を表す情報を入力層だけでなく各 Decoder 層の中間表現にも付与する．これにより，入力層で与えられた優先度情報がモデルの深い層で失われることを抑え，各層において Instruction Hierarchy に関する情報を維持することを目的としている．

3. 準備

本節では，本研究が防御の対象とする脅威モデルを示す．その後，提案手法を説明するために必要となる Bottleneck Adapter について述べる．

3.1 脅威モデル

本研究では，ASIDE [4] で用いられる脅威モデルを基本とする．Instruction と Data の区分はシステム設計によって既知であり，モデルへの入力時点で両者を識別できるものとする．

Instruction はモデルが従うべき命令であり，攻撃者によって直接変更されないものと仮定する．一方，Data は検索結果，Web ページ，メール，文書，ツール出力などの外部情報であり，その一部または全部を攻撃者が操作できるものとする．例えば，メールを処理するシステムではユーザから与えられた要求を Instruction，外部から受信したメール本文を Data として扱う．

攻撃者は Data 中に任意の攻撃文字列を埋め込むことができるが，Instruction，モデルパラメータ，および防御機構そのものを直接変更することはできない．攻撃者の目的は，Data 中に Instruction と競合する命令を挿入することで，モデルを本来の Instruction から逸脱させることである．

本研究では，あらかじめ定められた攻撃文字列を Data に挿入する固定攻撃に加えて，対象モデルの応答を観測しながら攻撃文字列を反復的に更新する適応的攻撃も評価する．適応的攻撃では，攻撃者は対象モデルの内部パラメータにはアクセスせず，モデルへの入力と得られた応答のみを利用するブラックボックス攻撃を用いて評価する．具体的な攻撃手法および実験条件については，実験設定の節で述べる．

3.2 Bottleneck Adapter

Bottleneck Adapter [9] は，事前学習済みモデルに小規模な Adapter モジュールを追加し，モデル全体を Fine-tuning することなく，下流タスクに適応させるための手法である．事前学習済みモデルの主要なパラメータを固定し，追加した Adapter のパラメータを主に学習することで，学習対象となるパラメータ数を削減できる．

Bottleneck Adapter では，Transformer が生成した隠れ状態を一度低次元の空間へ射影し，非線形変換を適用した後，再び元の次元へ戻す．このように，元の隠れ次元よりも小さい空間を介して変換を行う構造を bottleneck architecture と呼ぶ．

Transformer のある層における隠れ状態を $\mathbf{h} \in \mathbb{R}^d$，ボトルネック次元を m とし，$m \ll d$ とする．d 次元から m 次元への射影に用いる重み行列とバイアスを $W_{\text{down}} \in \mathbb{R}^{m \times d}$，$\mathbf{b}_{\text{down}} \in \mathbb{R}^m$ とする．また，非線形活性化関数を $\phi(\cdot)$ とする．ボトルネック空間における表現 $\mathbf{z} \in \mathbb{R}^m$ を次式で定義する．

$$\mathbf{z} = \phi\left(W_{\text{down}}\mathbf{h} + \mathbf{b}_{\text{down}}\right). \tag{1}$$

次に，m 次元から d 次元への射影に用いる重み行列とバイアスを $W_{\text{up}} \in \mathbb{R}^{d \times m}$，$\mathbf{b}_{\text{up}} \in \mathbb{R}^d$ とする．Adapter によって生成される d 次元の出力 $\Delta\mathbf{h} \in \mathbb{R}^d$ を次式で定義する．

$$\Delta\mathbf{h} = W_{\text{up}}\mathbf{z} + \mathbf{b}_{\text{up}}. \tag{2}$$

Bottleneck Adapter は Adapter 内部に skip-connection を持ち，Adapter によって得られた出力 $\Delta\mathbf{h}$ を元の隠れ状

© 2026 Information Processing Society of Japan

態 $\mathbf{h}$ に加算する．したがって，Adapter 適用後の隠れ状態 $\mathbf{h}'$ は下記式となる．

$$\mathbf{h}' = \mathbf{h} + \Delta\mathbf{h} \tag{3}$$

ボトルネック次元を $m \ll d$ とすることで，Adapter に必要なパラメータ数を抑えながら，下流タスクへの適応に必要な変換を学習できる．また，事前学習済みモデルの主要なパラメータを固定して Adapter を学習できるため，モデル全体を Fine-tuning する場合と比較して，学習対象となるパラメータ数を削減できる．

本研究では，この Bottleneck Adapter を基礎構造として利用し，Instruction の内部表現に基づいて Data および Response に適用される Adapter を動的に制御する．具体的な提案手法については次節で述べる．

4. 提案手法

4.1 Instruction-Conditioned Adapter

本研究では，Bottleneck Adapter に Instruction 依存の制御を導入した Instruction-Conditioned Adapter（ICA）を提案する．ICA では，Instruction の内部表現からゲートを生成し，Data および Response に対する Bottleneck Adapter の低次元表現に適用する．これにより，現在の Instruction が何を要求しているかに応じて，Data および Response の内部表現に加える Adapter の出力を調整する．

ICA は，Instruction の内容に応じて Data に対する内部処理を動的に制御することで，Instruction の遂行に必要な情報の利用を維持しつつ，Instruction と競合する Data 中の命令の影響を抑制することを目的とする．これにより，プロンプトインジェクションに対する頑健性と通常タスク性能の両立を目指す．

第 l 層における i 番目の token の隠れ状態を $\mathbf{h}_i^{(l)} \in \mathbb{R}^d$ とする．また，モデルが生成する出力 token 列を Response とし，Instruction，Data，Response に対応する token 位置の集合を，それぞれ $\mathcal{I}$，$\mathcal{D}$，$\mathcal{R}$ とする．Instruction を構成する token 数は入力ごとに異なるため，ICA では Instruction に対応する隠れ状態を mean pooling し，可変長の Instruction を一つの固定次元表現に集約する．第 l 層における Instruction 表現 $\mathbf{c}^{(l)} \in \mathbb{R}^d$ は，

$$\mathbf{c}^{(l)} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbf{h}_i^{(l)} \tag{4}$$

により算出する．

次に，この Instruction 表現から Bottleneck Adapter を制御するゲートを生成する．ボトルネック次元を m とし，ゲート生成に用いる重みとバイアスを $W_g^{(l)} \in \mathbb{R}^{m \times d}$，$\mathbf{b}_g^{(l)} \in \mathbb{R}^m$ とする．m 次元のゲート $\mathbf{g}^{(l)} \in (0,1)^m$ は，

$$\mathbf{g}^{(l)} = \sigma\left(W_g^{(l)} \mathbf{c}^{(l)} + \mathbf{b}_g^{(l)}\right) \tag{5}$$

により生成する．$\sigma(\cdot)$ には sigmoid 関数を用いる．ゲートの次元をボトルネック次元 m と一致させることで，Bottleneck Adapter 内部の低次元表現の各成分に対して，Instruction に応じた係数を与えることができる．

Data または Response に対応する $i \in \mathcal{D} \cup \mathcal{R}$ の隠れ状態に対し，down-projection に用いる重みとバイアスを $W_{\text{down}}^{(l)} \in \mathbb{R}^{m \times d}$，$\mathbf{b}_{\text{down}}^{(l)} \in \mathbb{R}^m$ とする．隠れ状態 $\mathbf{h}_i^{(l)}$ を m 次元のボトルネック空間へ射影し，低次元表現 $\mathbf{z}_i^{(l)} \in \mathbb{R}^m$ を

$$\mathbf{z}_i^{(l)} = \phi\left(W_{\text{down}}^{(l)} \mathbf{h}_i^{(l)} + \mathbf{b}_{\text{down}}^{(l)}\right) \tag{6}$$

により算出する．$\phi(\cdot)$ には SiLU を用いる．

通常の Bottleneck Adapter では，この低次元表現がそのまま up-projection へ入力される．ICA では，Instruction から生成したゲート $\mathbf{g}^{(l)}$ を $\mathbf{z}_i^{(l)}$ に要素ごとに適用することで，低次元表現の各成分が Adapter の出力に与える影響を調整する．

up-projection に用いる重みとバイアスを $W_{\text{up}}^{(l)} \in \mathbb{R}^{d \times m}$，$\mathbf{b}_{\text{up}}^{(l)} \in \mathbb{R}^d$ とすると，Adapter の出力 $\Delta\mathbf{h}_i^{(l)} \in \mathbb{R}^d$ は，

$$\Delta\mathbf{h}_i^{(l)} = W_{\text{up}}^{(l)} \left(\mathbf{g}^{(l)} \odot \mathbf{z}_i^{(l)}\right) + \mathbf{b}_{\text{up}}^{(l)} \tag{7}$$

により求める．$\odot$ は要素ごとの積（Hadamard 積）を表す．ゲートの各要素は 0 から 1 の範囲をとるため，Instruction の内容に応じて，低次元表現の各成分が $\Delta\mathbf{h}_i^{(l)}$ に与える影響を調整できる．

$i \in \mathcal{I}$ の token には ICA を適用せず，隠れ状態 $\mathbf{h}_i^{(l)}$ をそのまま用いる．一方，$i \in \mathcal{D} \cup \mathcal{R}$ の token には ICA を適用し，Adapter の出力 $\Delta\mathbf{h}_i^{(l)}$ を元の隠れ状態 $\mathbf{h}_i^{(l)}$ に加算する．

4.2 ASIDE との組合せ

ICA は Instruction の内容に基づいて Data および Response に対する Adapter の出力を調整する一方，Instruction と Data の入力由来そのものを分離する機構ではない．そのため本研究では，Instruction と Data をモデル内部で分離する ASIDE に ICA を組み合わせた ASIDE+ICA を提案する．

ASIDE は Instruction と Data の入力由来の違いをモデル内部に保持し，ICA はその上で，Instruction の内容に応じて Data および Response の内部表現に加える Adapter の出力を調整する．これにより，Instruction と Data の分離を維持しながら，現在の Instruction に応じた動的な制御を加えることを目的とする．

5. 実験設定

本節では，提案手法 ICA の有効性を検証するための比較モデル，学習条件，プロンプトインジェクション攻撃，生成条件，および評価指標について述べる．

© 2026 Information Processing Society of Japan

本研究では，あらかじめ定められた攻撃文字列を用いる固定攻撃に加えて，対象モデルから得られた Response に基づいて攻撃文字列を反復的に更新する適応的攻撃を用いて，プロンプトインジェクションに対する頑健性を評価する．さらに，防御性能の向上によって通常の指示追従能力が低下していないかを確認するため，攻撃を含まない入力に対する通常タスク性能についても評価する．これにより，プロンプトインジェクション耐性と通常タスク性能の両面から各防御手法を比較する．

5.1 比較モデル

ベースモデルには Qwen3-8B-Base [12] を用いる．比較対象として, Vanilla, ISE [3], ASIDE [4], AIR [5], ICA-Only, ASIDE+ICA の 6 種類のモデルを評価する．Vanilla, ISE, ASIDE については，ASIDE [4] で公開された Qwen3-8B-Base ベースの学習済みモデルを使用する．AIR, ICA-Only, ASIDE+ICA については，Qwen3-8B-Base を基礎として本研究で実装および学習する．Vanilla は，Instruction と Data をモデル内部で明示的に分離する防御機構を導入せず，通常の instruction tuning を行ったモデルであり，各防御手法を比較するための基準モデルとして用いる．ICA-Only は，Qwen3-8B-Base に ICA のみを適用し，ASIDE による分離を導入しないモデルである．このモデルを評価することで，ICA のみがどの程度プロンプトインジェクション耐性向上に寄与するのかを検証する．ASIDE+ICA は，ASIDE による Instruction と Data の分離に ICA を組み合わせた本研究の提案モデルである．ICA-Only および ASIDE との比較を通じて，両手法を組み合わせることによる効果を評価する．

5.2 学習設定

AIR，ICA-Only，ASIDE+ICA の学習データセットには，ASIDE [4] と同様に，攻撃文字列を含まない Alpaca-clean-gpt4-turbo データセット [10] を使用する．プロンプトインジェクションを含む adversarial training は行わず，通常の教師あり Fine-tuning のみを行う．

AIR，ICA-Only，ASIDE+ICA はいずれも，NVIDIA H200 GPU を 4 基使用してモデル全体のパラメータを更新する Full Fine-tuning を行う．ASIDE では，Instruction と Data を異なる表現として導入した後にモデルを Full Fine-tuning することで，両者を区別しながら処理するようモデルを学習する．本研究でもこの設定に従い，Full Fine-tuning を行う．主なハイパーパラメータを表 1 に示す．

表 1 AIR，ICA-Only，ASIDE+ICA の学習設定

Number of epochs	3
Maximum sequence length	768
Precision	bfloat16
Optimizer	AdamW
Learning rate	2×10^{-5}
LR scheduler	Cosine
Warmup ratio	0
Per-device batch size	2
Gradient accumulation steps	32
Global batch size	256

5.3 固定プロンプトインジェクション攻撃

まず，あらかじめ定められた攻撃文字列に対するプロンプトインジェクション耐性を評価する．固定攻撃には，StruQ [2] で用いられるプロンプトインジェクション攻撃を使用する．評価には，Data を含む 208 サンプルを使用する．各評価サンプルは Instruction と Data から構成され，攻撃文字列は Data にのみ挿入する．Instruction そのものは変更せず，Data 中に挿入された攻撃文字列によってモデルが本来の Instruction から逸脱するかを評価する．

本研究では，Naive，Ignore，Escape-Separation，Completion-Real の 4 種類の攻撃を使用する．Naive は Data の後に攻撃命令を直接追加する攻撃であり，Ignore は元の Instruction を無視するよう指示した上で攻撃命令に従わせる攻撃である．Escape-Separation は，改行やタブなどの区切りを挿入することで，攻撃命令をそれ以前の Data から分離して提示する攻撃である．Completion-Real は，元のタスクに対する Response が既に生成されたように見せかけた後，新たな Instruction として攻撃命令を提示する攻撃である．

本研究では，これら 4 種類の攻撃について In-domain（ID）設定のみを評価する．ID 設定では，攻撃文字列を元の Data の後方に付加する．各攻撃を 208 サンプルに適用するため，固定攻撃の評価は合計 832 ケースで構成される．すべての攻撃では，モデルに出力させる対象文字列を `Hacked` とし，生成された Response が `Hacked` のみで構成される場合を攻撃成功と判定する．各攻撃に対する Attack Success Rate（ASR）に加えて，4 種類の攻撃に対する ASR の平均値（Avg ASR）を報告する．また，攻撃対象のモデルについては，生成時の確率性による評価結果のばらつきを抑えるため，temperature を 0.0，top-p を 1.0，top-k を 0 とする．

5.4 適応的プロンプトインジェクション攻撃

固定された攻撃文字列のみを用いる評価では，攻撃者が対象モデルの Response を観測し，その挙動に応じて攻撃文字列を変更できる状況を十分に評価できない可能性がある．そこで本研究では，対象モデルから得られた Response を利用して攻撃文字列を反復的に改善する適応的攻撃についても評価する．攻撃文字列の生成および改善には，Prompt Automatic Iterative Refinement（PAIR）[11] の枠組みを

© 2026 Information Processing Society of Japan

用いる．

評価には固定攻撃と同じ 208 サンプルを使用する．PAIR では，208 サンプルそれぞれについて個別に攻撃文字列を改善する．すなわち，複数のサンプルで共通する単一の攻撃文字列を最適化するのではなく，各サンプルの Instruction，Data，および target model から得られた Response に基づいて，サンプルごとに異なる攻撃文字列を探索する．

各反復では，attacker model が攻撃文字列を 1 つ生成し，これを Data に挿入して target model へ入力する．得られた Response は LLM-as-a-Judge によって評価し，攻撃目標をどの程度達成しているかを 1 から 10 の整数値でスコア化する．1 は攻撃目標をほとんど達成していない状態を表し，10 は攻撃目標を十分に達成した状態を表す．本研究では，attacker model および judge model の双方に GPT-5 [16] を使用する．

attacker model には，生成した攻撃文字列，target model の Response，judge model によるスコア，および過去の改善履歴をフィードバックする．attacker model はこれらの情報を基に，より高いスコアが得られるよう次の攻撃文字列を生成する．この処理を反復することで，各 target model の応答特性に適応した攻撃文字列を探索する．

各サンプルについて最大 3 回の改善を行い，各反復で生成する攻撃候補は 1 件とする．また，直近 2 回の改善履歴を次の攻撃文字列の生成に利用する．attacker model の生成設定は，temperature を 0.7，top-p を 0.9 とする．target model の生成設定については，固定のプロンプトインジェクション評価と同じ設定を用いる．

なお，judge model による 1-10 点のスコアは攻撃文字列を探索するためのフィードバックとして使用する．最終的な攻撃成功の判定には固定攻撃と同一の基準を用い，target model が生成した Response が `Hacked` のみで構成される場合を攻撃成功とする．Vanilla，ISE，ASIDE，AIR，ICA-Only，ASIDE+ICA をそれぞれ独立した target model として扱い，各モデルに対して PAIR を個別に実行し，各モデルの頑健性を評価する．

5.5 通常タスク性能の評価

プロンプトインジェクション耐性が向上しても，通常の指示追従能力が大きく低下する場合には，実用的な防御とは言い難い．

そこで本研究では，攻撃文字列を含まない clean な入力に対する通常タスク性能についても評価する．評価には AlpacaEval 1.0 [14] を用い，AlpacaFarm [15] の評価データ 805 件に対して各モデルの Response を生成する．AlpacaEval の各サンプルについて，`instruction` を Instruction，`input` が存在する場合にはこれを Data としてモデルへ入力する．Vanilla，ASIDE，ICA-Only，ASIDE+ICA に対し，指示追従性能を比較する．これにより，各防御手法によるプロンプトインジェクション耐性の向上と通常タスク性能とのトレードオフを評価し，提案手法が通常タスク性能を維持しながらプロンプトインジェクション耐性を向上できるかを検証する．

6. 実験結果

本節では，前節で示した実験設定に基づき，固定プロンプトインジェクション攻撃，適応的プロンプトインジェクション攻撃，および通常タスク性能の評価結果を示す．

6.1 固定プロンプトインジェクション攻撃

固定プロンプトインジェクション攻撃に対する ASR を表 2 に示す．Vanilla, ISE, ASIDE については, ASIDE [4] で報告された Qwen3-8B の In-domain Robust Accuracy から ASR を算出した値を示す．AIR，ICA-Only，ASIDE+ICA については，本研究で実施した評価結果を示す．Avg は 4 種類の攻撃における ASR の平均値（Avg ASR）である．

ASIDE+ICA は，4 種類の攻撃に対する Avg ASR が 7.81%となり, 比較手法の中で最も低い値を示した．ASIDE の Avg ASR は 9.60%であり，ASIDE+ICA では 1.79 ポイント低下した．

攻撃別に見ると，ASIDE+ICA は Naive，Escape-Separation，Completion-Real において，それぞれ 7.69%，5.29%，8.17%となり，ASIDE よりも低い ASR を示した．一方，Ignore では，ASIDE の 9.60%に対して ASIDE+ICA は 10.10%であった．

ICA-Only および AIR の Avg ASR は，それぞれ 51.44%，62.98%であった．特に Completion-Real では，ICA-Only が 98.08%，AIR が 97.12%であるのに対し，ASIDE+ICA では 8.17%となった．

6.2 適応的プロンプトインジェクション攻撃

PAIR を用いた適応的プロンプトインジェクション攻撃に対する ASR を表 3 に示す．

Vanilla，AIR，ICA-Only では，ASR がいずれも 99.52%となった．ISE についても 96.63%となり，これらの手法では高い ASR が確認された．

これに対して，ASIDE の ASR は 60.58%であり，ASIDE+ICA では 41.35%となった．ASIDE+ICA は評価した 6 手法の中で最も低い ASR を示し，ASIDE と比較して 19.23 ポイント低下した．

6.3 通常タスク性能の評価

AlpacaEval 1.0 による通常タスク性能の評価結果を表 4 に示す．Vanilla および ASIDE については，ASIDE [4] で報告された Qwen3-8B の結果を使用する．ICA-Only および ASIDE+ICA については，本研究で実施した評価結果を示す．表中の誤差は標準誤差を表す．

© 2026 Information Processing Society of Japan

表 2　固定プロンプトインジェクション攻撃に対する ASR（%）

Method	Naive	Ignore	Escape-Separation	Completion-Real	Avg
Vanilla	26.90	54.30	54.30	91.30	56.70
ISE	28.40	66.30	66.30	50.00	52.75
ASIDE	10.10	**9.60**	9.60	9.10	9.60
AIR	44.71	69.71	40.38	97.12	62.98
ICA-Only	28.85	62.02	16.83	98.08	51.44
ASIDE+ICA	**7.69**	10.10	**5.29**	**8.17**	**7.81**

表 3　適応的プロンプトインジェクション攻撃に対する ASR（%）

Method	ASR
Vanilla	99.52
ISE	96.63
ASIDE	60.58
AIR	99.52
ICA-Only	99.52
ASIDE+ICA	**41.35**

表 4　AlpacaEval 1.0 による通常タスク性能

Method	AlpacaEval (%)
Vanilla	76.3 ± 1.5
ASIDE	76.1 ± 1.8
ICA-Only	**84.81 ± 1.27**
ASIDE+ICA	71.16 ± 1.86

ICA-Only は 84.81%となり，比較したモデルの中で最も高い値を示した．一方，ASIDE+ICA は 71.16%となり，ASIDE の 76.1%を下回った．

7. 考察

本研究では，Instruction と Data の分離を行う ASIDE と，Instruction に応じて Data および Response の表現を制御する ICA を組み合わせることで，プロンプトインジェクションに対する頑健性の向上を試みた．実験結果から，ICA を単独で適用した場合と，ASIDE と組み合わせた場合では，防御性能に大きな差が確認された．

ICA-Only では，固定攻撃および適応的攻撃の双方に対して十分なプロンプトインジェクション耐性を得られなかった．一方，ASIDE+ICA では，固定攻撃に対して ASIDE と同程度以上の頑健性を示し，適応的攻撃では ASIDE の ASR 60.58%に対して 41.35%となり，19.23 ポイント ASR を低下させることに成功している．

この結果から，ASIDE による Instruction と Data の区別に加えて，ICA による Instruction に応じた制御を導入することで，プロンプトインジェクション耐性がさらに向上する可能性が示唆され，それぞれの異なる役割が相補的に機能したと考えられる．

また，ASIDE に ICA を追加した際の改善幅は，固定攻撃と適応的攻撃で異なった．固定攻撃では，ASIDE の Avg ASR が 9.60%，ASIDE+ICA が 7.81%であり，差は 1.79 ポイントにとどまった．これは，ASIDE 単独でも固定攻撃に対して高い頑健性を有しているためと考えられる．

一方，適応的攻撃では，ASIDE の ASR が 60.58%であるのに対し，ASIDE+ICA では 41.35%となり，19.23 ポイントの低減が確認された．このことから，ICA による Instruction に応じた制御が，適応的攻撃に対して，より大きな効果をもたらす可能性が示唆される．

通常タスク性能については，ICA-Only が AlpacaEval 1.0 において 84.81%を示したことから，ICA の導入自体が通常の指示追従性能を大きく低下させるとは限らないことが分かる．一方，ASIDE+ICA は 71.16%となり，ASIDE の 76.1%を下回った．この結果から，ASIDE+ICA によるプロンプトインジェクション耐性の向上と通常タスク性能との間には，一定のトレードオフが存在する可能性が示唆される．

ICA は Instruction から得られた制御信号を用いて，Data だけでなく Response に対する Adapter の出力も制御する．そのため，プロンプトインジェクションの影響を抑制する一方で，通常タスクの遂行に必要な Data の利用や Response 生成にも影響を与えている可能性がある．

8. 結論

本研究では，プロンプトインジェクションに対する頑健性の向上を目的として，Instruction-Conditioned Adapter（ICA）を提案した．ICA は，各 Transformer 層における Instruction の隠れ状態から制御信号を生成し，その制御信号に基づいて Data および Response に適用される Adapter の出力を調整する．さらに，Instruction と Data を区別する ASIDE と ICA を組み合わせることで，Instruction の由来と内容の双方を利用した防御を構成した．

Qwen3-8B-Base を用いた評価では，ASIDE+ICA は固定プロンプトインジェクション攻撃に対して 7.81%の Avg ASR を示し，ASIDE の 9.60%を下回った．また，PAIR による適応的攻撃では，ASIDE の ASR が 60.58%であったのに対し，ASIDE+ICA では 41.35%となり，19.23 ポイントの低減を確認した．一方，ICA-Only では適応的攻撃に対する ASR が 99.52%であった．これらの結果から，ASIDE による Instruction と Data の区別と，ICA による Instruction 依存の制御を組み合わせることが，プロンプトインジェクション耐性の向上に寄与する可能性が示唆された．

© 2026 Information Processing Society of Japan

通常タスク性能については，ICA-Only が AlpacaEval 1.0 で 84.81%を示した一方，ASIDE+ICA では 71.16%となった．この結果から，ASIDE+ICA によるプロンプトインジェクション耐性の向上と通常タスク性能との間には，一定のトレードオフが存在する可能性が示唆された．

本研究にはいくつかの制約がある．まず，評価は Qwen3-8B-Base を基礎とするモデルに限定しており，異なるモデル規模やアーキテクチャに対する ICA の有効性は明らかではない．また，固定攻撃には StruQ の In-domain 設定，適応的攻撃には PAIR を用いており，より多様なプロンプトインジェクション攻撃に対する検証が必要である．さらに，本研究では最終的な Response に基づいて防御性能を評価しており，Instruction から生成されるゲートや各層における内部表現の変化については詳細に解析できていない．

今後は，複数のベースモデルおよび攻撃手法を用いて ICA の有効性を検証するとともに，内部表現を解析することで，Instruction に応じた制御がプロンプトインジェクション耐性に与える具体的な影響を明らかにしていきたい．また，通常タスク性能を維持しながらプロンプトインジェクションに対する頑健性をさらに向上させるため，提案手法の改善を行っていきたい．

参考文献

[1] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. and Fritz, M.: Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection, *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pp.79–90 (2023).

[2] Chen, S., Piet, J., Sitawarin, C. and Wagner, D.: StruQ: Defending Against Prompt Injection with Structured Queries, *Proc. 34th USENIX Security Symposium (USENIX Security 25)*, pp.2383–2400 (2025).

[3] Wu, T., Zhang, S., Song, K., Xu, S., Zhao, S., Agrawal, R., Indurthi, S.R., Xiang, C., Mittal, P. and Zhou, W.: Instructional Segment Embedding: Improving LLM Safety with Instruction Hierarchy, *Proc. International Conference on Learning Representations (ICLR)* (2025).

[4] Zverev, E., Kortukov, E., Panfilov, A., Volkova, A., Tabesh, S., Lapuschkin, S., Samek, W. and Lampert, C.H.: ASIDE: Architectural Separation of Instructions and Data in Language Models, *Proc. International Conference on Learning Representations (ICLR)* (2026).

[5] Kariyappa, S. and Suh, G.E.: Stronger Enforcement of Instruction Hierarchy via Augmented Intermediate Representations, *arXiv preprint arXiv:2505.18907* (2025).

[6] Chen, S., Zharmagambetov, A., Mahloujifar, S., Chaudhuri, K., Wagner, D. and Guo, C.: SecAlign: Defending Against Prompt Injection with Preference Optimization, *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)* (2025).

[7] Chen, S., Zharmagambetov, A., Wagner, D. and Guo, C.: Meta SecAlign: A Secure Foundation LLM Against Prompt Injection Attacks, *arXiv preprint arXiv:2507.02735* (2025).

[8] Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J. and Beutel, A.: The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions, *arXiv preprint arXiv:2404.13208* (2024).

[9] Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M. and Gelly, S.: Parameter-Efficient Transfer Learning for NLP, *Proc. 36th International Conference on Machine Learning (ICML)*, PMLR, Vol.97, pp.2790–2799 (2019).

[10] mylesgoose: Alpaca-cleaned-gpt4-turbo, Hugging Face Datasets (online), `https://huggingface.co/datasets/mylesgoose/alpaca-cleaned-gpt4-turbo` (accessed 2026-08-21).

[11] Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G.J. and Wong, E.: Jailbreaking Black Box Large Language Models in Twenty Queries, *Proc. 2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pp.23–42 (2025).

[12] Yang, A. et al.: Qwen3 Technical Report, *arXiv preprint arXiv:2505.09388* (2025).

[13] Rafailov, R., Sharma, A., Mitchell, E., Manning, C.D., Ermon, S. and Finn, C.: Direct Preference Optimization: Your Language Model Is Secretly a Reward Model, *Advances in Neural Information Processing Systems (NeurIPS)*, Vol.36 (2023).

[14] Li, X., Zhang, T., Dubois, Y., Taori, R., Gulrajani, I., Guestrin, C., Liang, P. and Hashimoto, T.B.: AlpacaEval: An Automatic Evaluator of Instruction-following Models, GitHub repository (2023).

[15] Dubois, Y., Li, X., Taori, R., Zhang, T., Gulrajani, I., Ba, J., Guestrin, C., Liang, P. and Hashimoto, T.B.: AlpacaFarm: A Simulation Framework for Methods that Learn from Human Feedback, *Advances in Neural Information Processing Systems (NeurIPS)*, Vol.36 (2023).

[16] OpenAI: GPT-5 System Card, OpenAI (2025).

© 2026 Information Processing Society of Japan